

CS2D: Documentación técnica

Facultad de Ingeniería de la Universidad de
Buenos Aires

Taller de Programación

Cátedra Veiga



Integrantes:

Alumno	Padrón
Fernandez, Facundo	109097
Molina , Taiel ALEXIS	109458
Rebollo, Matías Gabriel	109671
Rocha Diaz, Tomás	111085

Corrector: Capón, Mateo

Contenido

Introducción	2
Desarrollo	2
Arquitectura	2
Protocolo	4
Cierre del programa	5
Pruebas unitarias	5
Lógica de Juego	5
Fases de una ronda	5
Colisiones	6

Introducción

El presente documento busca explicar a grandes rasgos como ha sido modelado y desarrollado el videojuego Counter-Strike 2D, durante la cursada de la materia Taller de Programación I cátedra Veiga de la Facultad de Ingeniería Universidad de Buenos Aires. Antes de comenzar el documento se aclara que se necesitan conocimientos básicos sobre programación orientada a objetos (POO), multithreading y comunicación via sockets.

Desarrollo

El videojuego fue desarrollado con una arquitectura cliente-servidor en el cual el servidor es capaz de soportar múltiples clientes utilizando multithreading con mutexs como métodos de sincronización.

Por su parte el programa cliente consta de una interfaz gráfica, primero desarrollada con la librería QT y luego con SDL.

Arquitectura

La forma en que estos dos programas se comunican es a través de sockets. No se utilizó directamente un socket, si no una abstracción de más alto nivel, representado en una clase *Socket* diseñada por Di Paola Martín, docente y Jefe de Trabajos Prácticos de la materia.

El servidor al iniciarse, lanza otro hilo conocido como el *hilo aceptador* el cual se traba en su socket, esperando hasta que un nuevo cliente se una, y en caso de que esto suceda, lo acepta y le pasa el socket correspondiente con el que se va a comunicar al servidor, dejando al propio socket aceptador disponible para que otro nuevo cliente se una. También, al momento que se une un nuevo cliente, el servidor lanza un nuevo hilo para manejar sus peticiones.

Durante la fase en la que un cliente desea conectarse al servidor hasta que se une a una partida, el cliente corre en un único hilo y el servidor tiene un único hilo dedicado para él. Luego cuando ya se unió a una partida, además de crearse un propio hilo para la partida, se crea un hilo más del lado del servidor el cual, y dos hilos más del lado del cliente, generando la siguiente arquitectura.

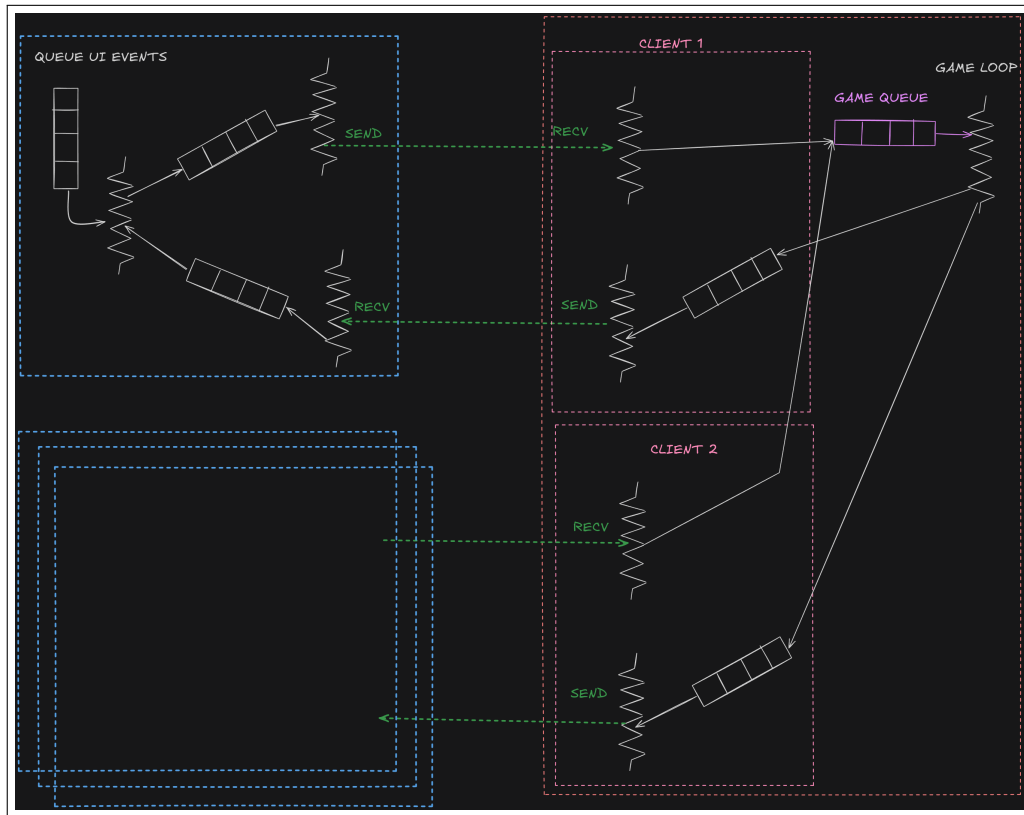


Figure 1: Arquitectura de hilos y queues durante una partida

Como se puede observar también, al momento de iniciar una partida, ambos hilos de los dos programas hacen uso de una queue que soporta multithreading, también diseñada por Di Paola, dejando finalmente este escenario:

- Cliente quiere enviar un comando a realizar en el juego: se detecta como un UI event en SDL, el UI loop lo detecta y lo envía a su queue sender, eventualmente el hilo sender poppeara ese comando de la queue y lo enviará via socket.
- Hilo receiver del lado del servidor recibe el comando enviado via socket, y lo envia a la queue del juego. Importante: cada juego tiene una única queue en la cual recibe las peticiones de todos los clientes unidos a ella.
- Luego de que el juego procesa toda la información de una iteración, envia el juego con la información actualizada a todos sus clientes: pusha a las queues de todos los clientes conectados el estado actual del juego (un snapshot).

- El hilo sender del cliente despoppea eventualmente la snapshot del juego en su hilo, y envia via socket el estado actual del juego.
- Finalmente el hilo receiver del lado del cliente recibe lo que describe el item anterior, y lo pushea a su queue para que eventualmente el hilo de la UI pueda renderizarse con la última actualización del juego.

Lo importante a saber de estos envíos es que ni el UI loop ni el game loop hacen pops ni pushes bloqueantes, porque de hacerlos, trabaríamos el juego en sí o la propia interfaz gráfica, dando un resultado indeseable. Luego los propios hilos senders/receivers de ambos lados sí hacen operaciones bloqueantes, tanto para leer o escribir en la queue, como para hacer esa misma interacción con el socket.

Protocolo

El protocolo utilizado para el juego es binario. A groso modo, cada mensaje del protocolo consta de un código identificador propio del mensaje en cuestión, y luego se envía la información pertinente. Mostrando un diagrama de clases del protocolo tenemos:

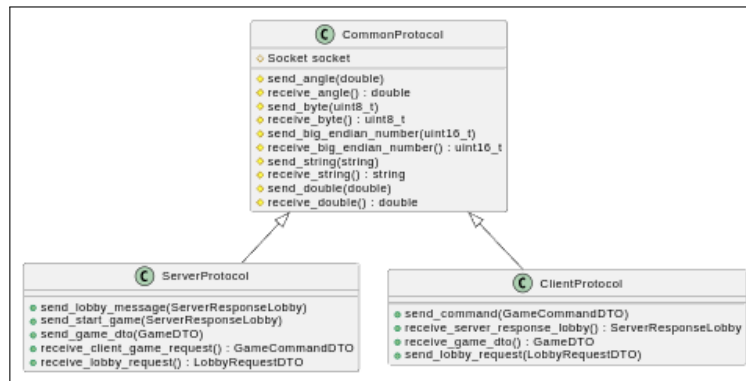


Figure 2: Protocolo del cliente y del servidor

Este como los siguientes diagramas UML mostrados son resumidos, porque por ejemplo en este caso tanto el protocolo servidor como el del cliente contienen numerosos métodos privados para facilitar el entendimiento del envío y recibimiento de mensajes via socket.

Como se puede observar, el CommonProtocol es quien posee al socket y además brinda distintos métodos que facilitan el envío de distintos tipos de datos. Luego las clases hijas que son quienes realmente se instancian, son las que conocen los structs de más alto nivel, pertenecientes al juego en sí, y quienes por dentro se encargan de enviar cada uno de sus atributos parseados a su código correspondiente en binario.

Cierre del programa

Los programas pueden cerrar de varias maneras, siendo esencialmente las siguientes:

- La partida termina: en este caso, los clientes verán durante unos segundos un mensaje final en el cual podrán observar las estadísticas de la partida. Pasado este tiempo, se procede a cerrar todos los sockets de ambos lados, y a parar ordenadamente los hilos. El programa cliente finalizará y se cerrará la ventana generada con SDL. En el momento en el que se una un nuevo cliente al servidor, se joinearan todos los hilos: los pertenecientes a todos los clientes de la partida, y el hilo de la propia partida en sí.
- Un cliente se desconecta durante una partida: se cerrarán los hilos y los sockets de la misma manera que lo comentado previamente, pero solo para este cliente, y además desde el juego no se enviarán más actualizaciones del juego a este cliente. En el caso de que haya quedado comandos o snapshots por enviar en las queues, recién cuando se vacíen, se cerrará todo. También cuando se una un nuevo cliente al servidor se joineará este hilo.
- Se cierra el server con la keyword designada (se ingresa una 'q' en la consola del servidor): igual que el primer item con la diferencia que se hace de manera forzada, cerrando todas las queues esperando a que se vacíen y finalmente cerrando todos los sockets abiertos y parando y joineando todos los hilos vivos (clientes y partidas).

Pruebas unitarias

Los unicos tests realizados en el proyecto son pruebas unitarias del protocolo, en los cuales con un mock socket se prueba que todo lo enviado por el cliente es recibido por el servidor de manera esperada, y viceversa. Se utilizó la librería GoogleTest para desarrollarlos.

Lógica de Juego

Cada juego de Counter-Strike 2D es una instancia de la clase CS2DGame que se encarga tanto de la comunicación con los jugadores conectados mediante recepción de comandos y broadcasteo de información, como de toda la lógica de juego (comunicándose con un objeto de clase GameWorld, que posee como atributo) y el desarrollo de las rondas y fases de cada ronda (comunicándose con un objeto subclase de GamePhase, según la fase de juego en la que se esté).

Fases de una ronda

La clase abstracta GamePhase implementa un método run() que será llamado por CS2DGame. Este método se encarga de simular el paso del tiempo en cada

iteración usando un objeto de clase Clock, haciendo que el juego ejecute comandos de distinta manera según la fase y broadcastee snapshots.

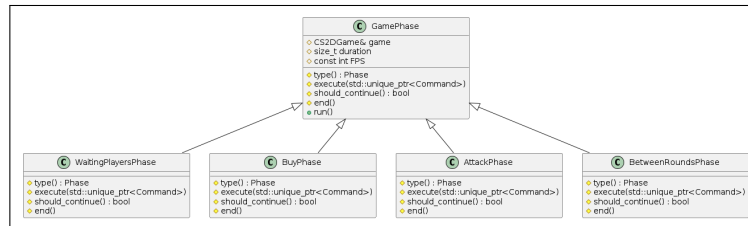


Figure 3: Fases de rondas del CS2D

Cada clase implementa de distinta manera cómo el juego debe ejecutar comandos (metodo execute), determina de distinta manera si la fase debe continuar y también cómo manejar el fin de una fase (conociendo cuál fase es la siguiente).

Todo esto permite que la implementación del run del hilo en donde corre la partida sea simplemente la siguiente:

```

1 void CS2DGame::run() {
2     while (should_keep_running()) {
3         if (this->current_round > ROUNDS) {
4             end_game();
5             break;
6         }
7         phase->run();
8     }
9 }
  
```

Ya que el run de cada phase se encargará de decirle al juego cuándo termina una ronda y qué hacer en cada momento de la misma.

Colisiones

La clase GameWorld tiene como atributo una lista de punteros a colisionables, para poder manejar de manera polimórfica los distintos objetos colisionables en el juego. Existen dos tipos de colisionables: objetos estáticos del mapa y los players.

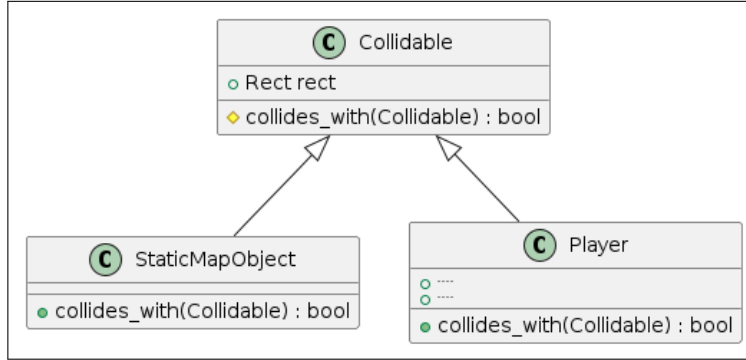


Figure 4: Colisionables

Cada vez que un jugador quiere "dar un paso" en alguna dirección, el GameWorld chequea que en la nueva posición que se le otorgará no haya ningún colisionable de la lista que colisione con él (obviamente, no chequea con el mismo jugador porque `collides_with` con daría true y nadie se podría mover).

El atributo `Rect` es un rectángulo que representa la hitbox de cada colisionable. El método `collides_with` devuelve true si el rect de un colisionable se interseca con otro, pero en el caso del jugador devuelve false si este está muerto.

Para los disparos dentro del juego, cada arma que dispara le pide al gameworld que calcule el impacto del disparo según la posición y dirección del jugador que dispara. Se llama para esto al método `GameWorld::calculate_shot`, que recorre todos los colisionables y calcula el primero que se encuentre en la trayectoria del disparo. No se busca ser muy verbosos en este documento, pero se hará un breve desarrollo sobre cómo se calcula si un disparo impacta o no en un colisionable:

Cada colisionable tiene una hitbox, que está formada por cuatro lados (cuatro segmentos). Un disparo tiene una posición origen y una dirección, por lo que existe una semirecta desde ese origen con la dirección dada. Entonces un disparo impacta en un colisionable si existe la intersección entre la semirecta y uno de los cuatro segmentos que hacen al rectángulo de la hitbox.

Sea la semirecta definida por la ecuación paramétrica:

$$R(t) = \mathbf{o} + \mathbf{d} \cdot t \quad \text{con } t \geq 0$$

y el segmento por:

$$S(u) = \mathbf{s} + \mathbf{v} \cdot u \quad \text{con } 0 \leq u \leq 1$$

donde: - $\mathbf{o}$ es el origen del disparo, - $\mathbf{d}$ es la dirección del disparo, - $\mathbf{s}$ es el punto inicial del segmento, - $\mathbf{v} = \mathbf{s}_{\text{end}} - \mathbf{s}_{\text{start}}$ es el vector director del segmento.

Buscamos los valores de t y u tales que:

$$\mathbf{o} + \mathbf{d} \cdot t = \mathbf{s} + \mathbf{v} \cdot u$$

$$\Rightarrow \mathbf{d} \cdot t - \mathbf{v} \cdot u = \mathbf{s} - \mathbf{o} \Rightarrow \mathbf{d} \cdot t + (-\mathbf{v}) \cdot u = \mathbf{r}, \quad \text{donde } \mathbf{r} = \mathbf{s} - \mathbf{o}$$

Esto es un sistema de 2 ecuaciones lineales con 2 incógnitas (t y u). Si trabajamos en $\mathbb{R}^2$, podemos resolverlo mediante productos cruzados:

$$t = \frac{\mathbf{r} \times \mathbf{v}}{\mathbf{d} \times \mathbf{v}}, \quad u = \frac{\mathbf{r} \times \mathbf{d}}{\mathbf{d} \times \mathbf{v}}$$

donde $\times$ denota el producto escalar en $\mathbb{R}^2$.

Para que haya intersección real y válida, se debe cumplir simultáneamente:

$$t \geq 0 \quad \text{y} \quad 0 \leq u \leq 1$$

Todo lo mencionado anteriormente se ve en los métodos `GameWorld::impacts` y `GameWorld::intersects_segment`. Si se cumple que hay intersección, se devuelve la distancia desde el origen de la semirrecta hasta la intersección y el punto de intersección. Si más de un segmento interseca con la semirecta, `GameWorld::impacts` devuelve la distancia mínima obtenida y el punto de intersección asociado a esta.